Lab No: 06

Student Name: Hania Nawab

Roll No: NUM-BSCS-2025-17

Task 1:	

Code:	#include <iostream>
	#include <string>
	#include <cctype>
	using namespace std;
	
	struct Stack{
	 char data;
	 Stack *next;
	};
	
	Stack *head = nullptr;
	
	void Push(char value)
	{
	 Stack *temp =new Stack();
	 temp->data = value;
	 temp->next = head;
	 head = temp;
	}
	
	char pop()
	{
	 if(head == nullptr)
	 {
	 return '\0';
	 }
	 else{
	 Stack *Todelet = head;
	 char value = Todelet->data;
	 head = head->next;
	 delete Todelet;
	 return value;
	 }
	}
	
	void clearStack()
	{
	 while(head != nullptr)
	 {
	 pop();
	 }

```
}

int checkpreferences(char a) {
    if (a == '^') return 3;
    if (a == '*' || a == '/') return 2;
    if (a == '+' || a == '-') return 1;
    return 0;
}

char seeOnly1() {
    if (head == nullptr) {
        return '\0';
    }
    return head->data;
}

bool isOpening(char ch)
{
    return (ch == '(' || ch == '{' || ch == '[');
}

bool isClosing(char ch)
{
    return (ch == ')' || ch == '}' || ch == ']');
}

bool isOperator(char ch)
{
    return (ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '^');
}

string reverseExp(string value)
{
    string rev = "";

    for (int i = value.length() - 1; i >= 0; i--) {
        char ch = value[i];

        if (ch == '(') { ch = ')'; }
        else if (ch == ')') { ch = '('; }
        else if (ch == '{') { ch = '}'; }
        else if (ch == '}') { ch = '{'; }
        else if (ch == '[') { ch = ']'; }
        else if (ch == ']') { ch = '['; }

        rev = rev + ch;
    }
    return rev;
}
```

```
string convert(string value, bool forPrefix) {
    string result = "";
    bool expectOperand = true;
    clearStack();

    for (int i = 0; i < value.length(); i++) {

        char ch = value[i];

        if (isspace((unsigned char)ch))
        {
            continue;
        }

        if (isalnum((unsigned char)ch))
        {
            if (expectOperand == false) { clearStack(); return "ERROR"; }
            result = result + ch;
            expectOperand = false;
        }
        else if (isOpening(ch))
        {
            if (expectOperand == false) { clearStack(); return "ERROR"; }
            Push(ch);
        }
        else if (isClosing(ch))
        {
            if (expectOperand == true) { clearStack(); return "ERROR"; }

            char target = '\0';

            if (ch == ')') {
                target = '(';
            }
            else if (ch == '}') {
                target = '{';
            }
            else if (ch == ']') {
                target = '[';
            }

            while (head != nullptr && !isOpening(seeOnly1())) {
                result = result + pop();
            }

            if (head == nullptr || seeOnly1() != target) { clearStack(); return "ERROR"; }
            pop();
        }
        else if (isOperator(ch))
        {
```

```
            if (expectOperand == true) { clearStack(); return "ERROR"; }

            while (head != nullptr) {
               char t = seeOnly1();
               bool popIt;

               if (forPrefix == true) {
               popIt = (checkpreferences(t) > checkpreferences(ch)) ||
(checkpreferences(t) == checkpreferences(ch) && ch == '^');
               }
               else {
                  popIt = (checkpreferences(t) > checkpreferences(ch)) ||
(checkpreferences(t) == checkpreferences(ch) && ch != '^');
               }

               if (popIt == false) {
                  break;
               }
               result = result + pop();
            }
            Push(ch);
            expectOperand = true;
         }
         else
         {
            clearStack();
            return "ERROR";
         }
      }

      if (expectOperand == true) { clearStack(); return "ERROR"; }

      while (head != nullptr) {
         if (isOpening(seeOnly1())) { clearStack(); return "ERROR"; }
         result = result + pop();
      }
      return result;
   }

   void conversionItPo(string value) {
      string result = convert(value, false);

      if (result == "ERROR") {
         cout << "Invalid Expression!" << endl;
      }
      else {
         cout << "Postfix Expression: " << result << endl;
      }
   }
```

```
void conversionItPr(string value) {
   string result = convert(reverseExp(value), true);

   if (result == "ERROR") {
      cout << "Invalid Expression!" << endl;
   }
   else {
      cout << "Prefix Expression: " << reverseExp(result) << endl;
   }
}

int main()
{
   int x;
   string value;

   do{
   cout << "1.Convert Infix to Prefix" <<endl;
   cout << "2.Convert Infix to Postfix "<<endl;
   cout << "0. Exit "<<endl;
   cin >> x;

   if (cin.fail())
   {
      cin.clear();
      cin.ignore(1000, '\n');
      x = -1;
   }
   else
   {
      cin.ignore(1000, '\n');
   }
   cout<<endl;

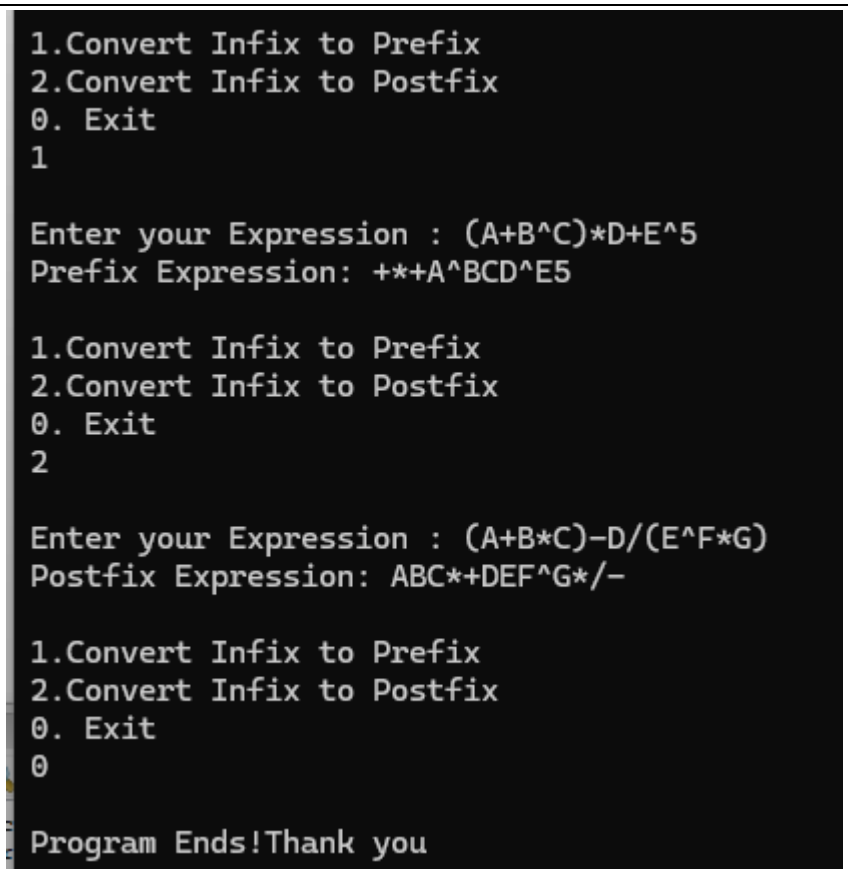
   if(x<0 || x>2)
   {
      cout<< "Invalid input! TRY AGAIN "<<endl;
   }
   if (x==1)
   {
     cout<<"Enter your Expression : ";
     getline(cin,value);
     conversionItPr(value);
     cout<<endl;
   }
   else if (x==2)
   {
     cout<<"Enter your Expression : ";
     getline(cin,value);
     conversionItPo(value);
```

```
    cout<<endl;
   }
   }
   while(x!=0);

   clearStack();
   cout<<"Program Ends!Thank you " <<endl;

   return 0;
}
```

Screenshots:



Task 2:

Code:

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

struct Stack{
 string data;
 Stack *next;
};

Stack *head = nullptr;
```

```
void Push(string value)
{
   Stack *temp =new Stack();
   temp->data = value;
   temp->next = head;
   head = temp;
}

string pop()
{
   if(head == nullptr)
   {
      return "";
   }
   else{
      Stack *Todelet = head;
      string value = Todelet->data;
      head = head->next;
      delete Todelet;
      return value;
   }
}

void clearStack()
{
   while(head != nullptr)
   {
      pop();
   }
}

bool isOperator(char ch)
{
   return (ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '^');
}

string convert(string value, bool toInfix) {

   clearStack();

   for (int i = value.length() - 1; i >= 0; i--) {

      char ch = value[i];

      if (isspace((unsigned char)ch))
      {
         continue;
      }
```

```
        if (isalnum((unsigned char)ch))
        {
            Push(string(1, ch));
        }
        else if (isOperator(ch))
        {
            if (head == nullptr) { clearStack(); return "ERROR"; }
            string a = pop();

            if (head == nullptr) { clearStack(); return "ERROR"; }
            string b = pop();

            if (toInfix == true) {
                Push("(" + a + ch + b + ")");
            }
            else {
                Push(a + b + ch);
            }
        }
        else
        {
            clearStack();
            return "ERROR";
        }
    }

    if (head == nullptr) { return "ERROR"; }
    string result = pop();

    if (head != nullptr) { clearStack(); return "ERROR"; }
    return result;
}

void conversionPrIt(string value) {
    string result = convert(value, true);

    if (result == "ERROR") {
        cout << "Invalid Expression!" << endl;
    }
    else {
        cout << "Infix Expression: " << result << endl;
    }
}

void conversionPrPo(string value) {
    string result = convert(value, false);

    if (result == "ERROR") {
        cout << "Invalid Expression!" << endl;
    }
```

```
    else {
        cout << "Postfix Expression: " << result << endl;
    }
}

int main()
{
    int x;
    string value;

    do{
    cout << "1.Convert Prefix to Infix" <<endl;
    cout << "2.Convert Prefix to Postfix "<<endl;
    cout << "0. Exit "<<endl;
    cin >> x;

    if (cin.fail())
    {
        cin.clear();
        cin.ignore(1000, '\n');
        x = -1;
    }
    else
    {
        cin.ignore(1000, '\n');
    }
    cout<<endl;

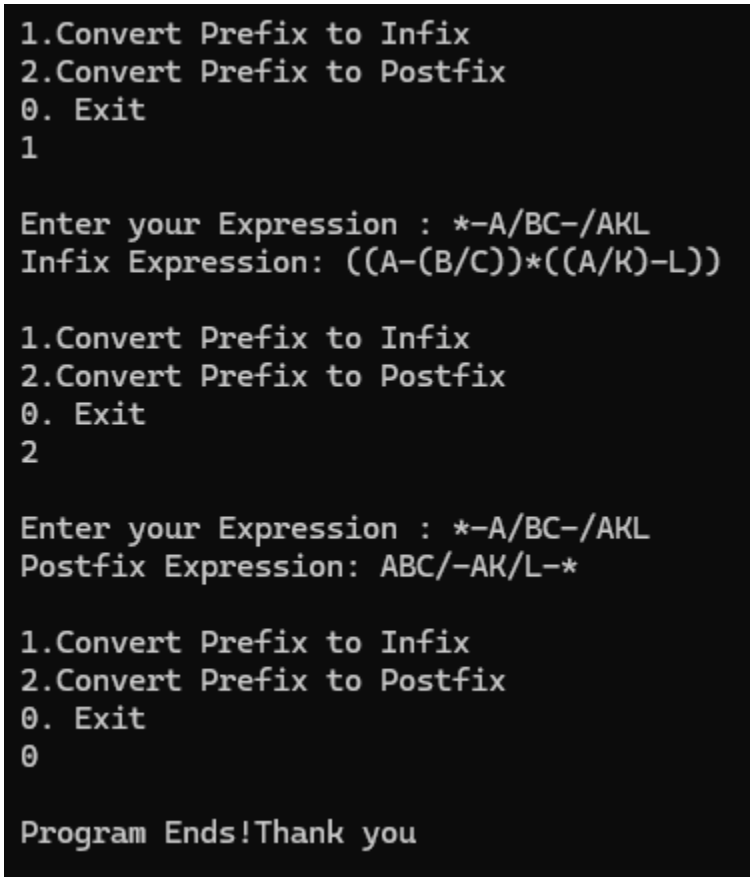
    if(x<0 || x>2)
    {
        cout<< "Invalid input! TRY AGAIN "<<endl;
    }
    if (x==1)
    {
      cout<<"Enter your Expression : ";
      getline(cin,value);
      conversionPrIt(value);
      cout<<endl;
    }
    else if (x==2)
    {
      cout<<"Enter your Expression : ";
      getline(cin,value);
      conversionPrPo(value);
      cout<<endl;
    }
    }
    while(x!=0);

    clearStack();
```

```
    cout<<"Program Ends!Thank you " <<endl;

    return 0;
}
```

Screenshots:



Task 3:

Code:

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

struct Stack{
 string data;
 Stack *next;
};

Stack *head = nullptr;

void Push(string value)
{
    Stack *temp =new Stack();
    temp->data = value;
```

```
    temp->next = head;
    head = temp;
}

string pop()
{
    if(head == nullptr)
    {
        return "";
    }
    else{
        Stack *Todelet = head;
        string value = Todelet->data;
        head = head->next;
        delete Todelet;
        return value;
    }
}

void clearStack()
{
    while(head != nullptr)
    {
        pop();
    }
}

bool isOperator(char ch)
{
    return (ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '^');
}

string convert(string value, bool toInfix) {

    clearStack();

    for (int i = 0; i < value.length(); i++) {

        char ch = value[i];

        if (isspace((unsigned char)ch))
        {
            continue;
        }

        if (isalnum((unsigned char)ch))
        {
            Push(string(1, ch));
        }
        else if (isOperator(ch))
```

```
        {
            if (head == nullptr) { clearStack(); return "ERROR"; }
            string b = pop();

            if (head == nullptr) { clearStack(); return "ERROR"; }
            string a = pop();

            if (toInfix == true) {
                Push("(" + a + ch + b + ")");
            }
            else {
                Push(string(1, ch) + a + b);
            }
        }
        else
        {
            clearStack();
            return "ERROR";
        }
    }

    if (head == nullptr) { return "ERROR"; }
    string result = pop();

    if (head != nullptr) { clearStack(); return "ERROR"; }
    return result;
}

void conversionPoPr(string value) {
    string result = convert(value, false);

    if (result == "ERROR") {
        cout << "Invalid Expression!" << endl;
    }
    else {
        cout << "Prefix Expression: " << result << endl;
    }
}

void conversionPoIt(string value) {
    string result = convert(value, true);

    if (result == "ERROR") {
        cout << "Invalid Expression!" << endl;
    }
    else {
        cout << "Infix Expression: " << result << endl;
    }
}
```

```
int main()
{
   int x;
   string value;

   do{
   cout << "1.Convert Postfix to Prefix" <<endl;
   cout << "2.Convert Postfix to Infix "<<endl;
   cout << "0. Exit "<<endl;
   cin >> x;

   if (cin.fail())
   {
      cin.clear();
      cin.ignore(1000, '\n');
      x = -1;
   }
   else
   {
      cin.ignore(1000, '\n');
   }
   cout<<endl;

   if(x<0 || x>2)
   {
      cout<< "Invalid input! TRY AGAIN "<<endl;
   }
   if (x==1)
   {
     cout<<"Enter your Expression : ";
     getline(cin,value);
     conversionPoPr(value);
     cout<<endl;
   }
   else if (x==2)
   {
     cout<<"Enter your Expression : ";
     getline(cin,value);
     conversionPoIt(value);
     cout<<endl;
   }
   }
   while(x!=0);

   clearStack();
   cout<<"Program Ends!Thank you " <<endl;

   return 0;
}
```

Screenshots:	